

Reviewer Pack — Real Rank of Moment Matrix for Random Max-CUT Instances is L...

Entry d71faf6d27d8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 05:26:50 UTC

Real Rank of Moment Matrix for Random Max-CUT Instances is Linear in n

Entry ID: d71faf6d27d8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 05:26:50 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry (Moment Matrices, Positivstellensatz)

Field B (complexity object): SOS Degree for Max-CUT

Statement:

For a random Max-CUT instance with n variables and m clauses, the real rank of its moment matrix is $\Theta(n)$, implying that the minimal SOS degree required to approximate the maximum cut is $\Theta(n)$.

Rationale (proposer's reasoning):

The real rank of the moment matrix captures the complexity of the polynomial system defining the Max-CUT instance. A linear real rank suggests that the SOS hierarchy requires linear degree to capture the solution, which could relate to the inherent structure of the problem.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c04a8724173d50f7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        # Find pivot
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        for j in range(i+1, m):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def matrix_rank(A):
    rank = 0
    A = gaussian_elimination(A)
    for row in A:
        if any(row):
            rank += 1
    return rank

def generate_maxcut_instance(n, m):
    variables = [f'x_{i}' for i in range(n)]
    clauses = []
    for _ in range(m):
        a, b = random.sample(variables, 2)
        polarity_a = random.choice([1, -1])
        polarity_b = random.choice([1, -1])
        clause = f"{polarity_a}*{a} + {polarity_b}*{b} >= 0"
        clauses.append(clause)
    return variables, clauses
```

```

def construct_moment_matrix(variables, clauses):
    n = len(variables)
    m = len(clauses)
    M = [[0] * (n + m) for _ in range(n + m)]

    # Identity matrix
    for i in range(n):
        M[i][i] = 1

    # Variables
    for i, var in enumerate(variables):
        for j, clause in enumerate(clauses):
            if var in clause:
                M[n + j][i] = 1

    return M

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m = max(1, int(n * (n - 1) / 8)) # Ensure at least one clause
        variables, clauses = generate_maxcut_instance(n, m)
        M = construct_moment_matrix(variables, clauses)

        rank = matrix_rank(M)
        results.append({
            "n": n,
            "m": m,
            "rank": rank
        })

    total_instances_tested = sum(len(result) for result in results)
    mean_rank = sum(result["rank"] for result in results) / total_instances_tested

    conjecture_holds = all(n <= result["rank"] <= 2 * n for result in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Real Rank of Moment Matrix",
        "metric_value": mean_rank,
        "instances_tested": total_instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_rank = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction:.2f}")
else:
    first_failing_seed = next(seed for seed, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96blab55.py", line 109, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96blab55.py", line 82, in run_trial
    rank = matrix_rank(M)
           ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96blab55.py", line 37, in matrix_rank
    A = gaussian_elimination(A)
        ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_96blab55.py", line 30, in gaussian_elimination
    factor = A[j][i] / A[i][i]
              ^^^^^
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing evaluation of rank properties | next: Fix numerical stability in Gaussian elimination or test with different parameter ranges

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	72314
2	propose	ollama_remote	qwen3:8b	0	0	110376
3	propose	ollama_remote	qwen3:8b	0	0	104273
4	preregistration	ollama_remote	qwen3:8b	0	0	24381
5	novelty	ollama_remote	qwen3:8b	0	0	20524
6	novelty	ollama_remote	qwen3:8b	0	0	16190
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14214
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11013
9	judge	ollama_remote	qwen3:8b	0	0	17690

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 390974 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d71faf6d27d8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d71faf6d27d8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d71faf6d27d8.tar.gz` (if generated)